

BCM STARTER KIT

Benutzerhandbuch

Governance & Lifecycle

Version 2.4.0

„BCM endet nicht mit dem Workshop.“

17. September 2026

Inhaltsverzeichnis

- 1** Review Center
- 2** Maßnahmenmanagement 2.0
- 3** BCM Timeline
- 4** Governance Dashboard

Anhang: Glossar

Review Center

Arbeitspaket AP1, Version 2.4.0.

Zweck

Das Review Center beantwortet für BCM-Verantwortliche eine einzige, konkrete Frage: **Was muss als Nächstes getan werden?** Es trifft dabei bewusst keine fachliche Aussage darüber, ob ein Review "ausreichend" durchgeführt wurde oder ob eine BCM-Entscheidung richtig war — das bleibt Sache des Anwenders.

Datenmodell

Jeder Review ist eine eigenständige Arbeits-/Historieninstanz in `STATE.reviews[]` (nicht nur eine Vorlage). Ein Review enthält:

Feld	Bedeutung
<code>id</code>	Eindeutige ID
<code>prozessId</code>	Bezug zur Prozessakte
<code>reviewart</code>	<code>prozess</code> <code>bia</code> <code>notbetrieb</code> <code>ressourcen</code>
<code>geplantAm</code>	Geplantes Datum
<code>gestartetAm</code>	Tatsächlicher Start (gesetzt beim Start)
<code>abgeschlossenAm</code>	Abschlussdatum (gesetzt beim Abschluss)
<code>verantwortlich</code>	Verantwortliche Person/Rolle
<code>status</code>	Nur <code>geplant</code> <code>in_bearbeitung</code> <code>abgeschlossen</code>
<code>ergebnis</code>	Freitext-Ergebnis/Feststellungen
<code>naechsterReviewAm</code>	Nächster geplanter Termin (manuell, solange AP2 keine Policy liefert)
<code>massnahmenIds[]</code>	Aus diesem Review entstandene Maßnahmen
<code>erstelltAm</code>	Anlagezeitpunkt

Reviewarten sind für Version 2.4 fest definiert (`REVIEW_ARTEN`): Prozessreview, BIA-Review, Notbetriebsreview, Ressourcenreview. AP2 ergänzt je Reviewart eine eigene Zykluspolicy (`process.reviewConfig`).

Fälligkeit ist berechnet, nicht gespeichert

Es gibt **keinen** gespeicherten Status "überfällig". `reviewDueInfo(r)` berechnet bei jedem Rendern aus `geplantAm` :

- `overdue` — geplantes Datum liegt in der Vergangenheit, Review nicht abgeschlossen
- `upcoming` — geplantes Datum liegt innerhalb der nächsten `REVIEW_UPCOMING_WINDOW_DAYS` (30) Tage
- `done` — Review ist abgeschlossen (unabhängig vom Datum nie überfällig)
- `ok` / `none` — weiter in der Zukunft bzw. kein Datum gesetzt

Priorisierung ("Was ist als Nächstes zu tun?")

`reviewCenterPriorityList()` liefert eine deterministisch sortierte Liste, ohne Score:

1. Überfällige Reviews, am längsten überfällig zuerst
2. Bald fällige Reviews, am nächsten fällig zuerst
3. Kritische Prozesse (`isProcessKritisch()`) ohne jeglichen Review-Eintrag

Jede Position in der Liste ist direkt aus dem Sortierkriterium erklärbar.

Review → Maßnahme

Aus einem Review kann direkt eine Maßnahme erzeugt werden (`App.reviewToMassnahme` / `App.confirmReviewToMassnahme`). Die neue Maßnahme wird in `STATE.massnahmen` angelegt und ihre ID in `review.massnahmenIds[]` verknüpft. AP3 (Maßnahmenmanagement 2.0) erweitert diese Verknüpfung um eine vollständig bidirektionale Herkunftskennzeichnung auf der Maßnahme selbst (`sourceType` / `sourceId` / `reviewId`).

Reviewhistorie in der Prozessakte

Der Maßnahmen-Tab jeder Prozessakte zeigt zusätzlich eine kompakte Reviewhistorie des jeweiligen Prozesses (`tabReviewHistoryCard()`) mit Link zurück ins vollständige Review Center — die eigentliche Bearbeitung bleibt bewusst zentral an einer Stelle.

Migration

Additive Schema-Migration 12 → 13 (`migrateV12ToV13`): ergänzt `STATE.reviews = []`, sofern nicht vorhanden. Bestehende 2.3.1-Dateien bleiben vollständig importierbar; kein Feld wird umbenannt oder entfernt.

Reviewzyklen je Reviewart (AP2)

Jede Reviewart besitzt eine eigene, optionale Zykluspolicy auf dem Prozess:

```
process.reviewConfig = {
  prozess:      { intervalType: '12m', intervalMonths: null } | null,
  bia:          { intervalType: '6m',  intervalMonths: null } | null,
  notbetrieb:   { intervalType: 'ereignisbezogen', intervalMonths: null } |
null,
  ressourcen:   { intervalType: '24m', intervalMonths: null } | null
}
```

Unterstützte `intervalType`-Werte: `3m`, `6m`, `12m`, `24m`, `individuell` (mit eigener `intervalMonths`-Zahl), `ereignisbezogen` (bewusst ohne Intervall). Der Default für jede Reviewart ist `null` — **keine Policy**.

`computeNextReviewDate(process, reviewart, fromDateStr)` ist die einzige Stelle, die aus einer Policy tatsächlich ein Datum macht, und verweigert das konsequent, wenn keine Policy definiert ist, die Reviewart ereignisbezogen ist, oder eine individuelle Policy keine gültige Monatsangabe hat. Es gibt keinen Fallback-Wert — in all diesen Fällen ist das Ergebnis `null`.

Nach Abschluss eines Reviews mit definierter Policy schlägt der Abschlussdialog den berechneten nächsten Termin vor und bietet an, den Folgereview (Status `geplant`) direkt anzulegen — die Entscheidung bleibt aber immer beim Anwender (Checkbox, standardmäßig aktiviert, aber abwählbar).

Editierbar ist die Policy kompakt direkt in der Prozessakte (`reviewConfigEditorHtml()`, im Maßnahmen-Tab neben der Reviewhistorie) — bewusst keine eigene Ansicht für vier Selects pro Prozess.

Migration (AP2)

Additive Schema-Migration 13 → 14 (`migrateV13ToV14`): ergänzt `process.reviewConfig` bei jedem Prozess, der es noch nicht besitzt — jede Reviewart startet bei `null`. Es wird nie eine Policy erfunden, auch nicht aus alten Freitext-Feldern.

Maßnahmenmanagement 2.0

Arbeitspaket AP3, Version 2.4.0.

Zweck

Erweitert das bestehende Maßnahmenmodell um Herkunft, Zeitachse und eine präzisere Wirksamkeitsprüfung — ohne eine zweite, parallele Maßnahmenstruktur einzuführen. Alle neuen Felder liegen additiv auf dem bestehenden `massnahmen[]`.

Neue Felder

Feld	Bedeutung
<code>sourceType</code>	<code>manuell</code> <code>review</code> <code>parkplatz</code> <code>qualitaet</code> <code>resilienz</code> <code>unbekannt</code> (nur Altdaten)
<code>sourceId</code>	Opaque Referenz auf den Auslöser (Review-ID, Parkplatz-ID, Regelcode ...)
<code>sourceLabel</code>	Menschenlesbare Kurzbeschreibung der Herkunft
<code>reviewId</code>	Nur gesetzt, wenn <code>sourceType==='review'</code> — zusätzlich zu <code>sourceId</code> , für direkte Verknüpfungsprüfung ohne Umweg über <code>sourceType</code>
<code>erstelltAm</code>	Anlagezeitpunkt (bei Altdaten leer, nie erfunden)
<code>abgeschlossenAm</code>	Wird beim ersten Wechsel auf <code>erledigt</code> automatisch gesetzt, danach nie überschrieben
<code>wirksamkeitPruefer</code>	Wer die Wirksamkeit geprüft hat (ergänzt das bestehende <code>wirksamkeitGeprueftAm</code> / <code>wirksamkeitErgebnis</code>)
<code>wiedervorlageAm</code>	Datum für eine erneute Prüfung/Wiedervorlage
<code>blockiertGrund</code>	Pflichtfeld-artig (Plausibilitätsprüfung), sobald Status <code>blockiert</code> ist

Statusmodell

`offen`, `geplant`, `in_arbeit` (Anzeige: "In Umsetzung"), `blockiert`, `erledigt`, `verworfen`. Der frühere Wert `zurueckgestellt` bleibt für Bestandsdaten gültig und lesbar (`ENUM_MASSNAHMEN_STATUS`), wird im Status-Dropdown für neue/bearbeitete Maßnahmen aber nicht

mehr angeboten (`MASSNAHMEN_STATUS_UI_OPTIONS`) — keine Breaking Change, aber ein klareres Zielmodell für neue Eingaben.

"Erledigt" bedeutet ausdrücklich nicht "wirksam". Diese Trennung existierte bereits vor 2.4.0 und wird durch AP3 nicht verändert, nur um den Prüfer (`wirksamkeitPruefer`) ergänzt.

`massnahmeIsOpen(m)` ersetzt mehrere zuvor verstreute `status==='offen' || status==='in_arbeit'` -Prüfungen (Dashboard, GF-Ansicht, Qualitätsprüfung) durch eine einzige Definition: offen ist alles außer `erledigt` und `verworfen` .

Herkunft — vier Entstehungspunkte

1. **Parkplatz** → **Maßnahme** (bestehende Funktion aus 2.3.0, jetzt mit Herkunftskennzeichnung: `sourceType: 'parkplatz'` , `sourceId` = Parkplatz- Eintrags-ID).
2. **Review** → **Maßnahme** (AP1, jetzt vollständig bidirektional): die Maßnahme erhält `reviewId` , das Review sein `massnahmenIds[]` -Eintrag.
3. **Qualitäts-/Konsistenzbefund** → **Maßnahme** (neu in AP3): ein "+ Maßnahme"-Knopf direkt in der Qualitätsprüfung. `sourceId` ist die Kombination aus Regelcode und Prozess-ID, da Befunde selbst keine eigene ID besitzen.
4. **Roter/gelber Resilienz-Check** → **Maßnahme** (bestehende automatische Erzeugung, jetzt mit `sourceType: 'resilienz'` , `sourceId` = Resilienz- Check-ID).

Ein Herkunfts-Filter und eine Herkunft-Spalte im Maßnahmenkatalog machen die Verteilung sichtbar (`ENUM_MASSNAHME_SOURCE_TYPE` , `MASSNAHME_SOURCE_LABELS`).

Plausibilitätshinweise (Dokumentationsqualität, keine fachliche Bewertung)

- Kein Verantwortlicher / kein Termin (bestehend)
- Termin überschritten (bestehend, `massnahmeIsOverdue()` — schließt seit AP3 `verworfen` - Maßnahmen aus, da eine verworfene Maßnahme nicht mehr "überfällig" sein kann)
- **Blockiert ohne Begründung** (neu, `massnahmeIsBlockedWithoutReason()`)
- **Erledigt ohne Wirksamkeitsnachweis** (bestehend als `massnahmeNeedsEffectivenessProof()` , jetzt zusätzlich in `validateMassnahme()`)
- **Review abgeschlossen, aber daraus entstandene Maßnahmen noch offen** (neu, `reviewsWithOpenFollowupMassnahmen()` , sichtbar im Review Center)

Keine dieser Prüfungen bewertet, ob die Blockade oder die Wirksamkeit fachlich zutreffend ist — nur, ob die Dokumentation dazu vollständig ist.

Migration

Additive Schema-Migration 14 → 15 (`migrateV14ToV15`): ergänzt die oben genannten Felder bei jeder bestehenden Maßnahme. Herkunft wird dabei niemals rückwirkend erfunden — Altdatensätze erhalten `sourceType: 'unbekannt'` statt z. B. `'manuell'`, und `erstelltAm` bleibt leer statt auf den Migrationszeitpunkt gesetzt zu werden.

BCM Timeline

Arbeitspaket AP4, Version 2.4.0.

Zweck

Eine chronologische, filterbare Übersicht fachlich relevanter BCM- Ereignisse. Kein Audit-Log, kein Event-Sourcing, keine Aufzeichnung jeder Feldänderung — nur die tatsächlich fachlich bedeutsamen Vorgänge.

Kein STATE.timeline[]

Die Timeline führt bewusst **keine eigene Ereignisliste**. Bevor irgendetwas neu gespeichert wurde, wurde zuerst geprüft, welche Ereignisse bereits aus vorhandenen Daten ableitbar sind — und das waren fast alle:

Ereignis	Quelle
Prozess angelegt	<code>process.createdAt</code>
Review geplant	<code>review.erstelltAm</code>
Review gestartet	<code>review.gestartetAm</code>
Review abgeschlossen	<code>review.abgeschlossenAm</code>
Maßnahme erstellt	<code>massnahme.erstelltAm</code> (AP3)
Maßnahme abgeschlossen	<code>massnahme.abgeschlossenAm</code> (AP3)
Wirksamkeit geprüft	<code>massnahme.wirksamkeitGeprueftAm</code>
Kritikalität geändert	Versionsdiff (<code>compareVersions()</code>)
MTA/RTO/RPO geändert	Versionsdiff
Notbetrieb wesentlich geändert	Versionsdiff (neu in AP4, siehe unten)
Kritische Ressourcen geändert	Versionsdiff
Version gespeichert	<code>STATE.versions[]</code>
Freigabe erzeugt	<code>STATE.versions[]</code> mit <code>source==='release'</code>

`buildTimelineEvents()` ist eine reine Funktion: liest `STATE`, gibt ein Array zurück, verändert nichts und speichert nichts. Ein Ereignis entsteht ausschließlich, wenn ein echter Zeitstempel vorhanden ist — ein nur geplanter Review (ohne `gestartetAm` / `abgeschlossenAm`) erzeugt genau ein Ereignis, nicht drei mit zwei erfundenen Daten.

Versionsdiffs statt Doppelpflege

Für "Kritikalität geändert", "MTA/RTO/RPO geändert" und "Kritische Ressourcen geändert" wird die bereits vorhandene `compareVersions()`-Logik wiederverwendet — angewendet auf jedes Paar **unmittelbar aufeinanderfolgender** Versionen (nie alle Paare, das würde Änderungen verzerrt vervielfachen oder Zwischenschritte verschlucken).

Für "Notbetrieb wesentlich geändert" wurde `compareVersions()` um einen Vergleich der operativen Kernfelder ergänzt (Auslöser, Entscheidung, die vier Reaktionsschritte, Rückkehr in den Normalbetrieb) — bewusst nicht jedes Freitextfeld, damit kleine Formulierungsänderungen die Timeline nicht überfluten. `compareVersions()` liefert außerdem seit AP4 je Prozessänderung dessen ID mit (vorher nur der Name) — für Drill-down und Prozessfilter benötigt, an der bestehenden Versionsvergleich-Ansicht ändert das nichts (die liest weiterhin nur `.name` / `.changes`).

Bewusst nicht umgesetzt: "Maßnahme gestartet" / "Maßnahme blockiert"

Für den Wechsel einer Maßnahme auf `in_arbeit` oder `blockiert` speichert das Datenmodell keinen eigenen Zeitpunkt. Ein Timeline-Ereignis dafür hätte einen Zeitpunkt erfinden müssen — das widerspricht "Ableitung vor Persistenz" und "keine Ereignisse für Funktionen erfinden, die im Produkt nicht existieren". Es wurde bewusst KEIN neues Zeitstempelfeld ergänzt, nur um dieses eine Ereignispaar zu ermöglichen (siehe `roadmap/DECISIONS.md`).

Filter und Drill-down

`timelineEvents({prozessId, type, from, to})` filtert und sortiert absteigend nach Zeitpunkt. Jeder Eintrag verlinkt (`timelineDrilldownHtml()`) in die passende bestehende Ansicht — Prozessakte, Review Center, Maßnahmenkatalog oder Versionshistorie — nie eine eigene Detailansicht.

Die Prozessakte (Maßnahmen-Tab) bietet zusätzlich einen kompakten "Timeline anzeigen"-Link, der direkt auf den jeweiligen Prozess filtert.

Governance Dashboard

Arbeitspaket AP5, Version 2.4.0.

Zweck

Das Governance Dashboard ist die operative Arbeits- und Steuerungsansicht, mit der ein BCM-Verantwortlicher innerhalb weniger Sekunden erkennt: "Das sind die Dinge, um die ich mich jetzt kümmern muss" — und bei jedem Punkt nachvollziehen kann, warum genau dieser Punkt angezeigt wird.

Es ist ausdrücklich **kein** Managementreport, Compliance-Dashboard, Auditbericht, Reifegradmodell, Risiko-Scoring-System oder BI-Dashboard.

Kein neues Datenmodell

`computeGovernanceDashboard()` ist eine reine Berechnungsfunktion. Sie wird bei jedem Rendern der Dashboard-Ansicht neu aufgerufen, liest ausschließlich `STATE`, verändert nichts und speichert nichts. Es gibt kein `STATE.dashboard[]`, keine gespeicherten Kennzahlen, keine gespeicherten Prioritäten — und keine neue Schema-Version (`CURRENT_SCHEMA_VERSION` bleibt bei 15).

Jede Aussage stützt sich auf bereits bestehende Funktionen:

Bereich	Wiederverwendete Funktion	Herkunft
Reviews	<code>reviewCenterData()</code>	AP1
Maßnahmen	<code>massnahmeIsOverdue()</code> , <code>massnahmeIsOpen()</code> , <code>massnahmeIsBlockedWithoutReason()</code> , <code>massnahmeNeedsEffectivenessProof()</code>	AP3
Qualität/Konsistenz	<code>qualityAndConsistencyCheck()</code>	vor 2.4
Änderungen/Freigabe	<code>buildTimelineEvents()</code>	AP4

Neu hinzugekommen sind ausschließlich kleine, rein filternde Aggregatoren (`massnahmeGovernanceData()`, `openManagementDecisions()`, `changesSinceLastRelease()`, `governanceQualityHighlights()`) — keiner von ihnen berechnet etwas, das nicht bereits an anderer Stelle existiert.

Die Priorisierung: 11 Stufen

`governancePriorityList()` ist eine feste, deterministische Abfolge von Filtern und Sortierungen — kein Score, keine KI, keine Zufallslogik. Gleiche Daten erzeugen immer dieselbe Reihenfolge (durch die Datensatz-ID als Tie-Breaker in jeder Sortierung).

1. Überfälliger Review eines kritischen Prozesses
2. Überfällige Maßnahme hoher Priorität
3. Blockierte Maßnahme mit hoher Relevanz (hohe Priorität ODER kritischer Prozess)
4. Kritischer Prozess ohne jegliche Reviewplanung
5. Bald fälliger Review eines kritischen Prozesses
6. Erledigte Maßnahme ohne dokumentierte Wirksamkeitsprüfung
7. Offener Managemententscheidungsbedarf (`massnahme.entscheidungsbedarf`)
8. Wesentlicher offener Konsistenz-/Qualitätsbefund (Konsistenzbefunde + Qualitätsbefunde der Stufe "Warnung")
9. Überfälliger Review eines sonstigen Prozesses
10. Sonstige überfällige Maßnahme
11. Sonstige fällige Governance-Aufgabe (bald fälliger Review eines sonstigen Prozesses, blockierte Maßnahme ohne hohe Relevanz, fällige Wiedervorlage)

Stufe 7 und die drei Bestandteile von Stufe 11 sind Ergänzungen zur 10-Punkte-Beispielreihenfolge aus der Aufgabenstellung — begründet in `roadmap/DECISIONS.md` (die Aufgabenstellung selbst verlangt, die Beispielreihenfolge gegen das tatsächliche Datenmodell zu prüfen und nicht blind zu übernehmen).

Jeder Eintrag enthält:

- **Aufgabe** (`title`) — kurzer Titel
- **Grund** (`reason`) — vollständiger, lesbarer Satz, nie nur "Hohe Priorität"
- **Zeitlicher Kontext** (`context`) — z. B. "14 Tage überfällig"
- **Deep Link** (`object` → `governanceDrilldownHtml()`) — direkter Klick in Review Center, Prozessakte, Maßnahmenkatalog, Qualitätsprüfung oder Timeline

Kompakte Detailkarten

Unterhalb der Prioritätsliste und der Kennzahlenreihe zeigt das Dashboard fünf kompakte Karten — jede eine reine Darstellung bereits berechneter Daten, nicht deren erneute Berechnung:

- **Review-Governance** — überfällige/bald fällige Reviews, kritische Prozesse ohne Reviewplanung (`reviewCenterData()`)

- **Maßnahmen-Governance** — überfällig, blockiert, offen mit hoher Priorität, ohne Wirksamkeitsnachweis, fällige Wiedervorlage (`massnahmeGovernanceData()`)
- **Änderungen seit letzter Freigabe** — siehe unten
- **Offene Managemententscheidungen** — siehe unten
- **Datenqualität & Konsistenz** — Warnungen und Konsistenzbefunde, nicht die vollständige Liste (dafür gibt es die Qualitätsprüfung)

Kritische Prozesse werden **nicht** als eigene Liste gezeigt — sie erscheinen ausschließlich dort, wo aus ihnen tatsächlich eine Governance-Aufgabe entsteht (Stufe 1, 3, 4, 5 der Prioritätsliste).

Änderungen seit letzter Freigabe

`changesSinceLastRelease()` ermittelt die zuletzt erzeugte Freigabe-Version (`STATE.versions[].source==='release'` , höchste `nr`) und listet fachlich relevante Timeline-Ereignisse (Kritikalität, MTA/RTO/RPO, Notbetrieb, kritische Ressourcen, abgeschlossene Reviews/Maßnahmen, geprüfte Wirksamkeit) seit deren Zeitpunkt — reine Wiederverwendung von `buildTimelineEvents()` (AP4), gefiltert nach Zeitstempel und Ereignistyp.

Existiert noch keine Freigabe, wird das transparent kommuniziert — es wird kein Referenzzeitpunkt erfunden (z. B. nicht "seit der ersten Version" oder "seit heute").

Offene Managemententscheidungen

`openManagementDecisions()` filtert Maßnahmen mit nicht-leerem `entscheidungsbedarf` , die noch nicht erledigt oder verworfen sind (`massnahmeIsOpen()`). Es wird keine neue Entscheidungs-Entität eingeführt — das Feld existierte bereits vor Version 2.4.

Empty States

- **Kein einziger Prozess dokumentiert:** die bestehende Willkommenskarte übernimmt diese Rolle bereits vollständig; das Governance Dashboard erscheint in diesem Fall gar nicht.
- **Prozesse vorhanden, aber keine Reviewplanung existiert:** eigene Meldung ("Für diese Prozesse ist noch keine Reviewplanung hinterlegt"), statt einer leeren "Keine dringenden Aufgaben"-Liste, die fälschlich "alles in Ordnung" suggerieren könnte.
- **Prioritätsliste tatsächlich leer, aber Reviewplanung existiert:** neutrale Meldung ("Aktuell keine dringenden Governance-Aufgaben erkannt") — keine positive Pauschalaussage wie "BCM vollständig aktuell".

Kennzahlen

Acht Kacheln, jede mit direktem operativem Nutzen: Reviews überfällig/bald fällig, Maßnahmen überfällig/blockiert, offene Wirksamkeitsprüfungen, kritische Prozesse ohne Reviewplanung, offene Managemententscheidungen, Änderungen seit letzter Freigabe. Keine künstlichen Prozentwerte, keine Reifegradzahl, kein Score — jede Zahl ist eine direkte Zählung eines bereits deterministisch klassifizierten Zustands.

Anhang: Glossar

Glossar

Kurzreferenz der zentralen Begriffe aus Version 2.4 — vollständige Erklärung jeweils im zugehörigen Kapitel.

Begriff	Bedeutung	Kapitel
Review	Konkrete Arbeits-/Historieninstanz einer Reviewart zu einem Prozess	1
Reviewart	Prozess-/BIA-/Notbetriebs-/Ressourcenreview	1
Fälligkeit/ Überfälligkeit	Aus <code>geplantAm</code> berechnet, nie gespeichert	1
Reviewzyklen-Policy	Optionales Intervall je Reviewart und Prozess	1
Herkunft (Maßnahme)	Woher eine Maßnahme entstand (Review/Parkplatz/Qualitätsbefund/Resilienz/manuell)	2
Wirksamkeitsprüfung	Separater Nachweis, dass eine erledigte Maßnahme tatsächlich wirkt	2
Wiedervorlage	Datum für eine erneute Prüfung einer offenen Maßnahme (<code>wiedervorlageAm</code>)	2
Blockiert ohne Begründung	Plausibilitätshinweis: Maßnahme mit Status <code>blockiert</code> , aber ohne <code>blockiertGrund</code>	2
Plausibilitätshinweis	Hinweis auf eine Dokumentationslücke (z. B. fehlende Begründung/Nachweis) — keine fachliche Bewertung der Maßnahme selbst	2
BCM Timeline	Chronologische, abgeleitete Ansicht fachlicher BCM-Ereignisse	3
Governance Dashboard	Zusammengeführte, priorisierte Arbeitsliste "Was ist als Nächstes zu tun?"	4
Priorisierungsstufe	Eine der 11 festen, deterministischen Rangstufen der Governance- Priorisierung	4
Deep Link	Direkter Klick-Link von einer Governance-Aufgabe in die zuständige bestehende Ansicht	4